Does an LLM Reorder Investment Candidates Better Than LambdaMART? A Same-Information Ranker Study on FAR-Trans

Student: Daria Babikova (Бабикова Дарья)
Team: [team name — TBD]; members: Кожокарь Алина, Целищева Ульяна, Бабикова Дарья
Date: 2026-11-13 · Assignment: Session 1 — Research Question and Minimal Study Plan (Define) · Track: Ranker · State: planned
Repository: https://github.com/dbabikova/RecSys-LLMs/tree/main/project_module2

Abstract

Problem. FAR-Trans [1] asks for a Top-10 list of financial assets per customer, scored by nDCG@10 against future purchases. LLM rankers reorder candidate lists well on movies and products [3], [4], but the only LLM study on FAR-Trans [2] reports an unranked top-3 hit rate, no nDCG and no same-information control. **Method.** I state one question — does zero-shot LLM reranking of a fixed top-20 list beat a LightGBM LambdaMART reranker that sees the same fields and the same candidates? — and plan a controlled comparison with four ablations, runtime and cost. **Result.** No model has been run. Data checks on a provisional split (cut 2022-05-31, 6-month label window) show why the question is open: an ItemKNN top-20 already contains 62.5% of users' new purchases but orders them at nDCG@10 = 0.290, so reordering alone has room; popularity reaches 0.193. Whether repeat purchases count as relevant changes the evaluated cohort from 1,132 to 2,590 users and the strongest simple baseline from 0.290 to 0.594. **Takeaway.** The comparison isolates the method (LLM vs. learned ranker) from information; the label rule must be fixed by the instructor before Session 2.

Index Terms: financial asset recommendation, FAR-Trans, LLM reranking, LambdaMART, nDCG@10, same-information control, leakage.

1. Introduction

Problem statement. FAR-Trans [1] holds 388,048 transactions of 29,090 retail customers of a Greek bank (2018-01 … 2022-11) over 806 stocks, bonds and mutual funds, plus customer profiles and daily prices. The shared task is one Top-10 list per evaluated customer, scored by nDCG@10 against assets bought in a future window; item scope, cut, users and label rule are pending, so every split here is an assumption.

Research question. *We test whether* **zero-shot LLM reranking of a fixed top-20 candidate list** *improves* **nDCG@10** *over* **a LightGBM LambdaMART reranker with the same user and asset information and the same candidates** *for* **the common FAR-Trans evaluation users.** Intervention: a frozen, prompted LLM orders the 20 candidates. Control: LambdaMART on the same candidates and fields. Outcome: nDCG@10 (runtime and cost secondary). Population: the common evaluation users, with pre-specified subgroups by history length (1 / 2–4 / ≥ 5 distinct assets). Track: Ranker; state: planned. No direction is assumed; a null or negative result will be reported.

Concrete example. One evaluated customer (Premium, risk *Aggressive*, capacity > €300k) bought two Greek corporate bonds in 2021. After the cut they bought the fund GRF000045002, which ItemKNN places at rank 12 of 20: the list contains the right asset, yet this customer's nDCG@10 is 0. Moving it to rank 3 gives $1/\log_2 4 = 0.5$; to rank 1 gives 1.0. The reranker decides only this order.

Research importance. LLM rankers are known to suffer position and popularity bias and to rely on item knowledge in their titles [3]. Financial assets have short, partly Greek names, thin metadata and a strongly concentrated demand (top-10 assets = 38.7% of buy pairs). Whether the LLM still adds ordering value beyond a learned ranker that uses the same fields is untested on this data; a positive and a negative answer are both new knowledge.

Business value. The decision that changes is the product owner's choice of ranking layer for an advisory recommendation rail: pay for GPU inference of an LLM ranker, or keep a CPU LambdaMART. A user study with FAR-Trans authors found that investors are poor judges of advice quality [5], so this choice cannot be made from how convincing LLM explanations look; it needs an offline comparison against real behaviour at equal information. No claim on returns, suitability or user acceptance is made: nDCG@10 measures ranking accuracy against observed purchases only.

Contributions.

- One RQ with a same-information control and ablations that separate semantic knowledge, personalisation and position bias (§3).
- A leakage audit of every input field, including two non-obvious future-information sources (§3.1).
- Provisional reference levels and a label-rule finding that the shared protocol must settle (§4).

2. Related Work

Study	Method	Data and evaluation	Limitations and connection to my RQ
Sanz-Cruzado et al., 2024, Fin-RecSys@IJCAI [1]	11 FAR algorithms: price-based regressors, Popularity, LightGCN, MF, UB-kNN, ARM, hybrids incl. LambdaMART	FAR-Trans; 61 bi-weekly cuts, 6-month test; Rel = buy; repeat pairs removed from test; nDCG@10, ROI@10 (LightGCN 0.340, Popularity 0.271)	No LLM. Source of the protocol, baselines and LambdaMART as an existing learned ranker

Study	Method	Data and evaluation	Limitations and connection to my RQ
Spadea & Seneviratne, 2025, ICAIF [2]	FLARKO: Qwen3 0.6–8B, LoRA + KTO, transactions and market serialised as knowledge graphs; LLM outputs ISINs	FAR-Trans, test 2021-12 … 2022-11; Pref@3 / Prof@3 / Comb@3 hit rates	Nearest prior work. No nDCG@10, unranked top-3, no runtime/cost, no same-information classical control
Hou et al., 2024, ECIR [3]	Zero-shot LLM ranks retrieved candidates from history; recency prompts, bootstrapping over permutations	ML-1M, Amazon Games; 20 candidates; NDCG@K	Position and popularity bias; weak use of history order. Motivates permutations and the no-history ablation
Yue et al., 2023, PGAI@CIKM [4]	LlamaRec: retriever top-20 → Llama-2-7B ranker with a verbalizer (one forward pass)	ML-100k, Beauty, Games; leave-one-out; NDCG/Recall@5,10; < 1 s per user	A ranker cannot recover items the retriever missed. Template for my fixed-candidate design and scoring

Table 1. Related studies I used for the RQ. Method = what the model does; Data and evaluation = datasets and metrics as reported; last column = gap or design consequence for this study.

Nearest prior work and gap. FLARKO [2] is the only LLM study on FAR-Trans I found. It answered whether a small fine-tuned LLM with graph context can name assets a customer buys and that are profitable (top-3 hit rate). It left open, under my condition, ranking quality of a Top-10 in nDCG@10 under the protocol of [1], and whether any gain comes from the LLM or from the extra information it receives. *We did not find a study reporting nDCG@10 for LLM reranking of a classical recommender's candidates on FAR-Trans against a learned ranker with the same information.* [3] and [4] show LLM ranking gains on items with informative titles; that this transfers to financial assets with thin metadata is not established.

Alternatives considered. (a) *Encoder track* — frozen LLM encoding of purchase history vs. a structured encoding of the same fields inside user-kNN (raised in the team, §5); clean control, but asset text is thin and user-kNN is a weak backbone (UB-kNN 0.160 in [1]). (b) *Hybrid* — deferred to Session 5 as the combination of (a) and this study. (c) LLM vs. original ItemKNN order only — rejected as the main comparison because it confounds method with information.

3. Method

3.1 Data, one real record and leakage

Source. FAR-Trans archive [6] (CC BY 4.0, SHA-256 d0d6cfe0…cfb6). Provisional split: prediction time t = 2022-05-31, labels = real buys in (t, 2022-11-30] — the latest cut that keeps the 6-month window of [1]. One record from the label window, IDs masked, other values verbatim:

```
source:    FAR-Trans (Univ. of Glasgow, researchdata.gla.ac.uk/1658), transactions.csv
record id: transactionID 1096…****
fields:    customerID, ISIN, transactionID, transactionType, timestamp, totalValue, units, channel,
marketID
values:    012E…****, GRF000045002, 1096…****, Buy, 2022-06-14, 49.865679, 2.655, Branch, AEDAK
window:    label window after t = 2022-05-31 — the target, never an input
```

Field (table)	Used by	Status	Timestamp in the example	Leakage rule
ISIN, type, date of past transactions	LLM + LambdaMART	observed; 38,019 reconstructed opening buys (ID < 0) flagged	2021-03-19, 2021-07-16	only rows < t
customerType	both	observed	snapshot 2021-03-19	latest snapshot ≤ t
riskLevel / investmentCapacity	both	observed here; predicted by the bank for 22–23% of rows	snapshot 2021-03-19	≤ t; predicted flag kept
lastQuestionnaireDate	not used	observed; 7,275 placeholders 2000-01-01 = missing	2022-02-16 (later than its snapshot)	use only if ≤ t
asset name, category, sub-category, sector, industry, market	both	observed; sector missing for 59/285 stocks and all funds	static	low risk
eligible universe (priced over whole test window)	protocol filter	derived	prices to 2022-11-29	future → never a feature
limit_prices.profitability	never	derived over 2018–2022	whole period	excluded

Table 2. Input fields for the example customer. Status: observed / predicted / missing as defined by the dataset; leakage rule = condition for use at prediction time t.

Two less obvious points: the protocol's eligibility filter uses post-t prices, so it must stay identical across arms and never become a feature; and profile snapshots are not strictly point-in-time (here the questionnaire date is later than its snapshot), so profiles are joined as-of t.

3.2 Arms, same information and the fixed candidate set

Candidates. ItemKNN (cosine on the binary customer × asset buy matrix before t), top-20, already-bought assets excluded; the list is fixed for every arm to isolate reranking, as the assignment allows. LightGCN may replace ItemKNN in Session 2 if its Recall@20 is higher; it is then frozen for all arms. **Arms.** R0 Popularity and R1 ItemKNN order (references); **C** LambdaMART (main control); **L** LLM reranker (intervention). **Same information.** C and L receive the same fields: history,

as-of profile, asset metadata, ItemKNN score and 6-month popularity. C gets them as features (shares of the history in the candidate's category / sector / industry / market, risk × category, history length, predicted flags, and a character-n-gram similarity between candidate and history names, so C also sees names); L gets them as text. C is trained on earlier cuts whose label windows end before t, with early stopping on the cut t − 6 months, over 5 seeds.

LLM ranker (proposal, fixed in Session 2). Open-weights Qwen3-8B (Hugging Face revision hash to be reported), local inference, frozen, temperature 0. Scoring follows [4]: candidates labelled A–T, score = next-token logit of each label, one forward pass, so no invented assets and no sampling noise. Against position bias [3], each user is scored under 3 fixed random permutations and the logits are averaged. If the GPU budget is short, Qwen3-4B is the fallback ([2] found 1.7–4B competitive on FAR-Trans).

3.3 Measures, contribution check and risks

Measures. nDCG@10 with binary relevance and IDCG over min(|Rel|, 10), mean over users, official unrounded value; per-user paired differences L − C with a 95% bootstrap interval (analysis only); C as mean ± sd over 5 seeds; L spread over the 3 permutations. Secondary: end-to-end recommendation-generation time per user on the same machine, GPU-hours and prompt tokens, share of the 10 most popular assets in the lists. No LLM judge is used; the only ground truth is purchases.

Ablation	What it isolates
L without asset names (ISIN codes, metadata kept)	semantic and world knowledge about assets; also the hindsight probe below
L without purchase history	personalisation vs. the LLM's popularity prior
L without profile	contribution of risk level and investment capacity
L with candidates in ItemKNN order, one pass	position bias and anchoring on the generator
C without the name-similarity feature	whether names help even a classical ranker

Table 3. Ablations; each removes or changes one component and leaves the candidates, users and metric unchanged.

Risks. *Contamination:* the LLM was pretrained after 2022 and may know what happened to Greek assets in late 2022; a gain that disappears without names is read as possible hindsight, not skill, and the model's knowledge cutoff is reported. *Privacy:* data are pseudonymised; customer IDs never enter a prompt; inference is local. *Ethics:* ranking accuracy is not suitability or profit, and no advice claim is made. *Data quality:* joint accounts (52,308 rows share a transaction ID across two customers), Greek-language fund names, sub-category conflicts.

Feasibility. Prompts ≈ 0.8–1.2k tokens; main L run plus ablations ≈ 18k forward passes in the worst case (2,590 users), a few GPU-hours on one GPU — to be measured on 100 users first. Session 2: frozen protocol, generator, evaluation code, LambdaMART, L on 100 users. Session 3: full C vs. L. Session 4 (optional): LoRA tuning of the ranker. Session 5: ablations, failure analysis, draft. Session 6: final run and defence. Tools: Python, pandas, LightGBM, PyTorch/transformers, GPU from Colab/Kaggle/university [confirm].

4. Experiments

No model comparison has been run; §4.3 is the planned evaluation. §4.1–4.2 report data checks produced by committed scripts in the repository; all split-dependent numbers are on the provisional single cut and are sanity checks, not results.

4.1 Data accounting

Quantity	Value	Quantity	Value
transactions (buy / sell)	388,048 (228,913 / 159,135)	customers / traded assets	29,090 / 320 of 806
reconstructed opening buys (ID < 0)	38,019 (35,620 dated 2018-01-02)	real buys	190,894
repeat share of real buys	68.7%	top-10 assets' share of buy pairs	38.7%
eligible universe at t	511 assets	test buyers / with history before t	3,668 / 3,145 (523 cold)
evaluated users, rule A (new pairs)	1,132 (1.67 relevant each)	evaluated users, rule B (all buys)	2,590 (2.84 relevant each)

Table 4. Dataset and provisional-split counts. Rule A = FAR-Trans protocol [1] (repeat pairs dropped, owned assets excluded from lists); rule B = every purchase in the window is relevant. Customer-segment counts match [1] exactly (e.g. Mass 18,610).

4.2 Provisional reference levels

Rule	Ranker	Users	nDCG@10	Recall@10	Recall@20	≥ 1 hit in top-20
A	Popularity (6 months)	1,132	0.193	0.374	0.542	63.3%
A	ItemKNN (candidate generator)	1,132	0.290	0.452	0.625	71.6%
B	Popularity (6 months)	2,590	0.286	0.514	0.644	78.3%
B	Repeat own purchases + popularity	2,590	0.594	0.729	0.821	90.9%

Table 5. Single cut t = 2022-05-31. Recall@20 is the ceiling of any reordering of a top-20 list; nDCG@10 of the generator's own order is the level the rerankers must beat. For scale, [1] reports Popularity 0.271 and LightGCN 0.340 averaged over 61 cuts.

Under rule A the candidate set holds 62.5% of relevant assets but ranks them at 0.290, so a reranker has room. Under rule B a rule that repeats the customer's own purchases reaches 0.594: the LLM would have to beat a strong non-learned baseline, and a repeat-aware baseline becomes mandatory. The rule therefore changes the experiment, not only the numbers.

4.3 Planned evaluation

Arms R0, R1, C, L on the official users, cut and rule; report nDCG@10 overall and per subgroup, Recall@20 ceiling, runtime, cost, popularity share; then the five ablations of Table 3. Expected reading: L > C — the LLM adds ordering value beyond a learned ranker with the same fields (checked against hindsight via the no-name ablation); L ≈ C — no measurable gain at this metadata richness, a useful negative result for the GPU-vs-CPU decision; L < C — zero-shot ranking is hurt by bias in this domain, which motivates tuning in Session 4.

4.4 Verification

- **My own check.** For the example customer I opened `customer_information.csv` from the archive and confirmed the profile row: Premium, Aggressive, CAP_GT300K, questionnaire 2022-02-16, snapshot 2021-03-19 — the only snapshot ≤ t. [Add: the label row found in transactions.csv by its values — 2 rows, the second is the joint-account copy; the two history rows; the hand nDCG of the example: rank 12 → 0, rank 3 → $1/\log_2 4 = 0.5$]
- **Supporting re-run (not my own check).** `scripts/verify_example.py`, written by the AI assistant, recomputes the example from the raw CSVs without the project's helper code: 12 / 12 checks match, including the ItemKNN rank 12 and the identical top-20 order.

5. Discussion

[Draft — to be updated after the team discussion; report only what actually happened.]

Options considered. (1) My question: LLM reranking vs. LambdaMART on a fixed top-20 (Ranker). (2) Raised in the team: *does frozen LLM encoding of the purchase-history profile improve nDCG@10 over a structured encoding of the same features with an unchanged user-kNN?* (Encoder). (3) Hybrid of (1) and (2) with an encoder/ranker ablation. Also weighed: label rule A vs. B; ItemKNN vs. LightGCN as generator; LLM vs. original order as the main comparison (dropped, §2). [who proposed (2); other options raised]

Agreement and unresolved differences. [Fill in.] Proposed common ground: one shared protocol, generator and evaluation code; (1) and (2) are complementary and meet in a Hybrid at Session 5. Open: the label rule (question sent to the instructor [confirm]); who takes which track.

My contribution. Chose the Ranker track and the main control; formulated and fixed the RQ; confirmed the example profile in the raw files. [Add what I proposed, checked or argued in the meeting; mark joint ideas as joint.]

My position and reasons. I adopt (1). Reasons from the data and the literature: the comparison changes one thing only (the ordering method) with information held equal; the candidate list holds 62.5% of relevant assets at nDCG@10 0.290 (rule A), so ordering has measurable headroom; the gap to [2] is concrete (no nDCG@10, no same-information control). For (2), the structured code already contains sector labels, so a null effect is likely unless the LLM knows more than the metadata, and user-kNN is a weak backbone (0.160 in [1]).

Threats to validity. Single provisional cut (the official protocol may differ); thin, partly Greek metadata; popularity concentration that can make any ranker look like popularity; possible hindsight knowledge of the LLM; joint accounts counting one purchase twice; profile snapshots not strictly point-in-time.

6. AI Usage Disclosure

AI tools used. Claude (Anthropic), model Claude Opus 5.5, in a claude.ai session with code execution and access to my GitHub repository; it ran Python (pandas, numpy, matplotlib), web search and page fetches, and committed to `project_module2/` under my Git identity with a Co-Authored-By line. [number of messages / tool calls from the export]. Part of the repository history (commits 7526c1a, 515f2d9, c3af84b) comes from a parallel branch of the same chat. [state whether that branch's export is attached]

How AI was used. Planning the stages from the assignment; data scripts (`inspect_schema.py`, `make_data_example.py`, `eda.py`, `candidate_recall.py`, `make_ranker_example.py`, `verify_example.py`) and their reports; the literature search and summaries; comparisons of the three tracks; drafting this report. I chose the track, the RQ and the main control, and brought the team's Encoder question into the session for comparison.

What I personally verified. The example customer's profile row in the raw `customer_information.csv` (values in §4.4). [Add each further check with its numbers.]

What I trusted without full verification. All counts in Tables 4–5 and §3.1 (script output, not recomputed by me); the 12/12 re-run; the summaries of [1]–[5] until I open each paper myself [confirm opened]; the budget estimate in §3.3.

Session log. `babikova_session1_session.json` [Claude export of this session — confirm file name] is attached unmodified as the audit trail of the prompts, code, tool calls and results above.

References

[1] J. Sanz-Cruzado, N. Droukas, and R. McCreadie, "FAR-Trans: An investment dataset for financial asset recommendation," in Proc. IJCAI-2024 Workshop on Recommender Systems in Finance (Fin-RecSys), Jeju, South Korea, 2024. [Online]. Available: https://arxiv.org/abs/2407.08692

[2] F. Spadea and O. Seneviratne, "Aligning language models with investor and market behavior for financial recommendations," in Proc. 6th ACM Int. Conf. AI in Finance (ICAIF '25), Singapore, 2025. [Online]. Available: https://arxiv.org/abs/2510.15993

[3] Y. Hou, J. Zhang, Z. Lin, H. Lu, R. Xie, J. McAuley, and W. X. Zhao, "Large language models are zero-shot rankers for recommender systems," in Proc. ECIR 2024, LNCS, vol. 14609, pp. 364–381, 2024. [Online]. Available: https://doi.org/10.1007/978-3-031-56060-6_24

[4] Z. Yue, S. Rabhi, G. de Souza Pereira Moreira, D. Wang, and E. Oldridge, "LlamaRec: Two-stage recommendation using large language models for ranking," in PGAI Workshop at CIKM 2023, 2023. [Online]. Available: https://arxiv.org/abs/2311.02089

[5] T. Takayanagi, K. Izumi, J. Sanz-Cruzado, R. McCreadie, and I. Ounis, "Are generative AI agents effective personalized financial advisors?," in Proc. SIGIR 2025, 2025. [Online]. Available: https://arxiv.org/abs/2504.05862

[6] University of Glasgow, "FAR-Trans dataset," research data repository, 2024, CC BY 4.0. [Online]. Available: https://doi.org/10.5525/gla.researchdata.1658

[7] LLM4RecSys course, "Lecture 07. LLM-Based Recommendation and FAR-Trans Project," HSE University, 2026 (course slides).